

프롤로그

서버 뼈대에 막히다

오픈이는 이제 막 자바(Java)를 배우기 시작한 학생입니다. 이번 학기 과제인 **게시판 웹 서버**를 만들기 위해 호기롭게 코드를 작성하기 시작했지만, 몇 줄 쓰지도 못하고 벽에 부딪혔습니다.

문제는 게시판 기능이 아니라 그 이전 단계에 있었습니다. 웹 서버를 직접 만들려면 외부와 통신하기 위한 작업부터, 들어온 요청을 해석하고 주소에 맞는 코드를 찾아 실행하는 처리까지 모두 구현해야 합니다.

중요한 게시글 기능은 시작도 못했는데, 서버 뼈대를 만들기 위한 작업만 산더미같이 쌓여 있었습니다. 오픈이는 멍하니 모니터를 보며 한숨을 쉬었습니다.

이거 뼈대만 만들다가 과제는 손도 못 대겠는데...

다음 날, 오픈이는 같은 수업을 듣는 선배에게 고민을 털어놓았습니다. 선배는 오픈이가 지금까지 쓴 코드를 훑어보더니 말했습니다.

선배 : "이런 걸 처음부터 직접 만드는 사람은 없어요. **웹 서버에 공통으로 필요한 뼈대는 프레임워크에 이미 갖춰져 있거든요.** 자바로 웹 서버를 만들면 보통 **스프링(Spring)** 을 써요. 스프링으로 시작해서 게시판 기능만 구현해 봐요."

선배의 말을 들은 오픈이는 스프링에 대해 알아보기로 했습니다. 스프링은 자바를 기반으로 복잡한 애플리케이션을 쉽게 만들 수 있도록 돕는 프레임워크로, 외부 요청을 처리하는 서버의 기본 뼈대를 이미 갖추고 있습니다. 따라서 개발자가 모든 흐름을 직접 통제할 필요가 없습니다. 클래스와 메서드에 약속된 표식만 붙여 두면, 요청이 들어왔을 때 **스프링이 표식을 읽어 알맞은 코드를 찾아 실행합니다.**

개발자가 메서드를 직접 호출하던 방식과 달리, 프레임워크가 개발자의 코드를 호출하도록 흐름이 뒤집힌 이 원리를 **제어의 역전(IOC)** 이라고 합니다. 서버 뼈대를 만드는 일을 스프링에 위임하고 나니, 오픈이는 핵심인 게시판 기능에만 온전히 집중할 수 있게 되었습니다.

게시판을 완성하다

오픈이는 컴포넌트 스캔으로 대상을 찾고 리플렉션으로 메서드를 호출하는 스프링의 동작 방식을 이해했습니다. 이제 남은 건 기능을 만들어보는 일이었습니다.

다음 날 저녁, 새 프로젝트를 만들기 위해 노트북을 열었습니다. 첫 번째 목표는 게시판이었습니다.

스프링으로 게시판의 기본 기능을 만들어 보자.

오픈이는 게시판 서버를 완성된 화면 없이 순수 데이터만 주고받는 **API 서버** 형태로 만들기로 했습니다. 일반적인 웹사이트는 서버가 화면을 완성해 전달합니다. 반면 API 서버는 데이터만 응답하며, 브라우저나 휴대폰 같은 디바이스에서 이 데이터를 처리합니다. 서버는 화면과 상관없이 데이터만 응답하기 때문에, 브라우저와 휴대폰처럼 서로 다른 디바이스도 하나의 서버를 함께 사용할 수 있습니다. 오픈이는 API 서버를 만드는 방식 중 가장 널리 쓰이는 **REST API**를 선택했습니다.

그다음 게시글을 데이터베이스에 저장하기 위해 테이블을 설계했습니다. 그런데 스프링은 데이터를 자바 객체로 다루고, 객체와 테이블은 구조가 다릅니다. 그래서 **객체의 값을 테이블에 옮겨 담으려면 매핑 작업이 필요합니다**. 이 과정을 쉽게 처리하기 위해 스프링에서는 **JPA**라는 기술을 사용합니다. JPA는 이 매핑 정보를 바탕으로 저장과 조회에 필요한 SQL을 생성합니다.

JPA는 **영속성 컨텍스트**라는 공간을 통해 객체를 관리합니다. 영속성 컨텍스트는 **애플리케이션과 데이터베이스 사이에서 객체를 보관하고, 객체에 생긴 변경을 모아 데이터베이스에 반영합니다**.

오픈이는 스프링과 JPA를 활용해 게시판의 기본 기능을 구현해 냈습니다. 학기 과제로 기획했던 게시판이 첫 동작을 시작한 순간이었습니다.

예외처리를 하다

게시판 기본 기능을 완성한 오픈이는 다음 날 뿌듯한 마음으로 선배에게 결과물을 보여주었습니다. 선배는 게시글 목록과 상세 조회 기능을 차례로 확인하더니, 이번에는 게시글 번호를 999로 바꿔 다시 조회했습니다. 그러자 상태 코드는 여전히 200(성공)이었지만, 정작 응답에 담긴 게시글 데이터는 null이었습니다.

데이터베이스에 없는 게시글인데 오류가 나지 않고 왜 성공으로 응답하지?

선배 : "데이터베이스에서 조회된 게시글이 없으면 null 값을 응답하는데, 이 null 값을 서버에서 처리하지 않고 응답하니까 사용자는 정상적인 응답을 받게 될 거예요. 만약 이 상태에서 오류가 발생하면, 사용자는 자신의 문제인지 서버의 문제인지 원인을 알 수 없는 오류를 보게 되겠죠. 그래서 서버에서는 이런 **예외(Exception)**가 발생할 경우에 대비해 미리 예외 처리를 해야 해요. 그래야 어떤 사유로 요청이 실패했는지 클라이언트에게 명확히 알려줄 수 있어요."

선배가 말을 이어갔습니다.

선배 : "그리고 지금은 서버가 객체를 그대로 응답하는 구조라서 사용자에게 불필요한 정보까지 전달되고, 보안에도 취약해요. 그래서 **DTO**라는 데이터 전송을 위한 객체를 만들고, 이 DTO 타입으로 요청과 응답을 보내야 해요."

조언을 들은 오픈이는 예외 처리에 대해 알아보았습니다. **예외 처리는 정상적인 흐름을 이어갈 수 없는 상황이 발생했을 때, 프로그램이 멈추는 대신 오류의 원인을 사용자에게 안내하는 역할을 합니다.** 오픈이는 학습한 내용을 바탕으로 프로젝트의 구조를 수정하기 시작했습니다. 먼저 예외가 발생했을 때 한 곳에서 처리할 수 있도록 전역 예외 처리기를 추가했습니다. 그리고 DTO를 만들어 사용자에게 필요한 정보만 응답할 수 있도록 구조를 변경했습니다.

이제 오픈이의 서버는 오류가 발생하면 미리 정해 둔 상태 코드와 오류 메시지를 사용자에게 돌려줍니다.

요청한 사람을 식별하다

지금까지 오픈이가 구현한 게시판의 게시글은 누가 작성했는지 알 수 없는 익명 게시글이었습니다. 다음은 회원 기능을 추가해 인증 게시판을 만드는 것입니다. 인증 기능을 추가하기 위해 공부하던 오픈이는 로그인 상태를 관리하는 방식이 크게 두 가지라는 것을 알게 되었습니다. 하나는 서버가 사용자의 로그인 정보를 저장하는 **상태 유지(Stateful)** 방식이고, 다른 하나는 로그인 정보를 저장하지 않는 **무상태(Stateless)** 방식입니다.

둘 중에 어떤 방식으로 인증을 만들어야 하지?

오픈이는 선배에게 어떤 방식을 선택하는 것이 좋을지 물어보았습니다.

선배 : "상태 유지 방식은 서버가 로그인 정보를 직접 관리하니까, 필요할 때 특정 사용자를 바로 로그아웃시키는 것처럼 통제하기 쉬워요. 대신 **사용자가 늘어날수록 서버가 기억해야 할 데이터도 많아져서 부담이 커지고**, 서버가 여러 대가 되면 그 정보를 서로 공유해야 하는 문제가 생겨요."

선배 : "반면 무상태 방식은 **서버가 로그인 정보를 기억할 필요가 없어서 사용자가 늘어나도 부담이 크지 않고**, 서버가 몇 대든 요청만 검증하면 돼요. 대신 한 번 발급한 인증 정보는 만료되기 전까지 서버가 중간에 무효로 만들기 어렵다는 단점이 있어요. 지금 만드는 REST API 방식에는 무상태 방식이 더 적합할 것 같아요. **JWT**라는 토큰을 사용하면 무상태 방식으로 인증을 할 수 있어요."

선배의 조언을 들은 오픈이는 JWT에 대해 알아보았습니다. **JWT는 사용자 인증 정보를 자체적으로 담고 있는 토큰입니다**. 그래서 서버는 로그인 상태를 따로 기억하지 않아도, 요청에 포함된 JWT만으로 사용자를 확인할 수 있습니다. 오픈이는 이 방식을 적용해 **사용자 인증 기능과 게시글 권한 처리 기능**까지 구현했습니다.

게시글과 댓글을 연결하다

인증 게시판을 완성한 오픈이는 마지막으로 댓글 기능을 추가하기로 했습니다. 댓글은 게시글 상세 화면에서 게시글과 함께 조회되어야 하는데, 게시글 하나에 여러 개의 댓글이 포함되므로 둘 사이를 연결할 **외래키는 댓글 테이블이 가져야 합니다**.

이때 데이터베이스는 외래키로 조인을 하면 양쪽 테이블의 데이터를 가져올 수 있지만, **JPA는 테이블이 아닌 객체 단위로 동작하기 때문에** 게시글→회원, 댓글→게시글 같이 **상대 객체를 필드로 가진 쪽에서만 접근할 수 있습니다**. 그래서 현재 구조에서는 게시글에서 댓글 데이터를 조회할 수 없습니다.

지금 구조에서는 게시글에서 댓글 정보를 가져올 수가 없는데, JPA를 안 쓰고 직접 SQL을 사용해야 하나?

오픈이는 선배에게 지금까지 만든 코드를 보여 주며 고민을 이야기했습니다. 선배는 게시글과 댓글 객체를 차례로 살펴보더니 말했습니다.

선배 : "실무에서는 크게 두 가지 방법을 써요. **첫 번째는 쿼리를 두 번으로 나눠서 조회하는 거예요**. 먼저 게시글을 기준으로 회원 정보를 조회하고, 그다음 댓글을 기준으로 댓글 작성자 정보를 추가로 가져오는 거죠. 그리고 두 데이터를 DTO에서 합치는 방식이에요."

오픈이 : "그럼 두 번째 방법은요?"

선배 : "**JPA의 양방향 매핑을 쓰는 거예요**. 게시글에 댓글 필드를 추가하면 게시글에서도 댓글을 참조할 수 있게 돼요. 이렇게 하면 양쪽에서 참조가 가능하기 때문에 조인을 통해 한 번에 데이터를 가져올 수 있어요."

선배의 조언을 들은 오픈이는 양방향 매핑에 대해 알아보았습니다. **양방향 매핑은 반대쪽 객체에도 참조를 위한 필드를 추가해 두 객체가 서로를 참조하게 만드는 방식입니다**. 그래서 게시글 쪽에 댓글 필드를 추가하면 게시글에서도 댓글을 참조할 수 있습니다.

이렇게 객체끼리 참조하게 되면 **연관된 데이터를 언제 가져올지도 정해야 합니다**. 항상 연관된 데이터를 함께 가져오면 당장 필요 없는 데이터 때문에 불필요한 쿼리가 추가로 실행될 수 있습니다. 그래서 JPA는 조회하는 시점에 연관된 데이터를 같이 가져오는 **즉시 로딩(Eager Loading)** 과, 실제로 사용하는 시점에 가져오는 **지연 로딩(Lazy Loading)** 중 하나를 선택할 수 있게 지원합니다. 오픈이는 지연 로딩을 선택하고, 게시글과 댓글을 함께 보여줘야 하는 상세 조회에서는 조인을 사용해 한 번에 가져오도록 했습니다.

이렇게 양방향 매핑과 지연 로딩을 활용해 게시판에 댓글 기능을 추가했습니다.

오픈이는 완성한 게시판을 선배에게 보여주었습니다.

오픈이 : "처음엔 과제를 끝낼 수 있을지 걱정이 앞섰는데, 결국 완성하게 되네요."

선배는 미소 지으며 격려했습니다.

선배 : "처음부터 완벽한 사람은 없어요. 하나씩 차근차근 풀어나가다 보면 이렇게 멋지게 완성할 수 있는 거죠."